

Sistema de Detección y Segmentación de Objetos en Tiempo Real mediante Ajuste Fino de YOLOv8

Trabajo Práctico Final — Deep Learning
Maestría en Inteligencia Artificial

Federico Morán

Julio de 2026

Resumen

Este trabajo presenta el desarrollo de un sistema completo de visión por computadora capaz de detectar y segmentar, en tiempo real, ocho clases de objetos de entorno doméstico y de oficina. Se construyó un conjunto de datos de 5.000 imágenes y 23.363 instancias anotadas a partir de COCO 2017, se realizó el ajuste fino (*fine-tuning*) de un modelo de segmentación de instancias YOLOv8n-seg preentrenado, y se evaluó el resultado sobre una partición de prueba con las métricas estándar del área. El modelo ajustado alcanzó un mAP@50-95 de 0,524 para detección (cajas) y de 0,452 para segmentación (máscaras), lo que representa una mejora relativa del 30 % y el 32 % respectivamente frente al modelo preentrenado de referencia evaluado sobre las mismas imágenes. El sistema de inferencia, implementado sobre OpenCV con captura de cámara web, opera a 17 cuadros por segundo sobre una CPU de consumo, y alcanza 35 cuadros por segundo al optimizar el modelo con OpenVINO, lo que valida la viabilidad de la segmentación de instancias en tiempo real sobre hardware no especializado.

1. Introducción

La detección y segmentación de objetos son dos tareas centrales de la visión por computadora. La **detección** localiza cada objeto mediante una caja delimitadora y le asigna una clase; la **segmentación de instancias** va un paso más allá y delimita, píxel a píxel, la silueta exacta de cada objeto individual. Su combinación en un único sistema que opere en *tiempo real* habilita aplicaciones de robótica, asistencia, control de inventario y análisis de video, entre otras, pero impone un compromiso exigente entre precisión y latencia, en particular cuando la inferencia debe ejecutarse sobre hardware de consumo sin aceleración dedicada.

El objetivo general de este trabajo es desarrollar un sistema completo de visión por computadora que detecte y segmente objetos en tiempo real. Los objetivos específicos son: (i) construir un conjunto de datos supervisado a partir de COCO 2017; (ii) ajustar un modelo de segmentación de instancias mediante transferencia de aprendizaje; (iii) evaluarlo con las métricas estándar del área (mAP e IoU, tanto para cajas como para máscaras), incluida la comparación contra el modelo preentrenado de referencia; y (iv) implementar la inferencia en tiempo real sobre una computadora personal, con captura de cámara web.

El alcance se acotó a ocho clases de objetos de entorno doméstico y de oficina (**person**, **cell phone**, **cup**, **bottle**, **laptop**, **keyboard**, **mouse**, **book**), seleccionadas con un criterio de aplicación: todas pueden exhibirse físicamente frente a una cámara web durante la demostración del sistema.

El documento se organiza como sigue. La Sección 2 describe la construcción del conjunto de datos. La Sección 3 presenta la arquitectura, el preprocesamiento, el aumento de datos y la configuración de entrenamiento. La Sección 4 reporta los experimentos y resultados. La Sección 5 describe el sistema de inferencia en tiempo real y sus mediciones de desempeño. La Sección 6 expone las conclusiones.

2. Conjunto de datos

2.1. Fuente y selección

Se utilizó **COCO 2017** (*Common Objects in Context*) [1], el conjunto de datos de referencia para detección y segmentación de instancias: sus 80 categorías de objetos cotidianos están anotadas a nivel de instancia mediante polígonos, lo que habilita ambas tareas. En lugar de descargar el conjunto completo (~ 19 GB), se realizó una *descarga selectiva* mediante la biblioteca FiftyOne [5]: se obtuvieron 5.000 imágenes del split de entrenamiento que contienen al menos una instancia de las clases objetivo, junto con sus anotaciones de segmentación, con selección aleatoria y semilla fija para garantizar la reproducibilidad.

Las anotaciones se filtraron para conservar únicamente las instancias de las ocho clases de interés, lo que define con precisión el alcance del problema: los objetos de las restantes 72 categorías pasan a formar parte del fondo. El subconjunto resultante contiene **23.363 instancias anotadas**, con un promedio de 4,67 objetos por imagen (mediana 3, máximo 47).

La Figura 1 muestra la distribución de instancias por clase. El desbalance es pronunciado: **person** concentra el 74,2 % de las instancias, mientras que **mouse** apenas reúne 139 ejemplos. Este desbalance, heredado de la estadística natural de COCO, no se corrigió de forma artificial, pero se tiene presente al interpretar las métricas por clase de la Sección 4.

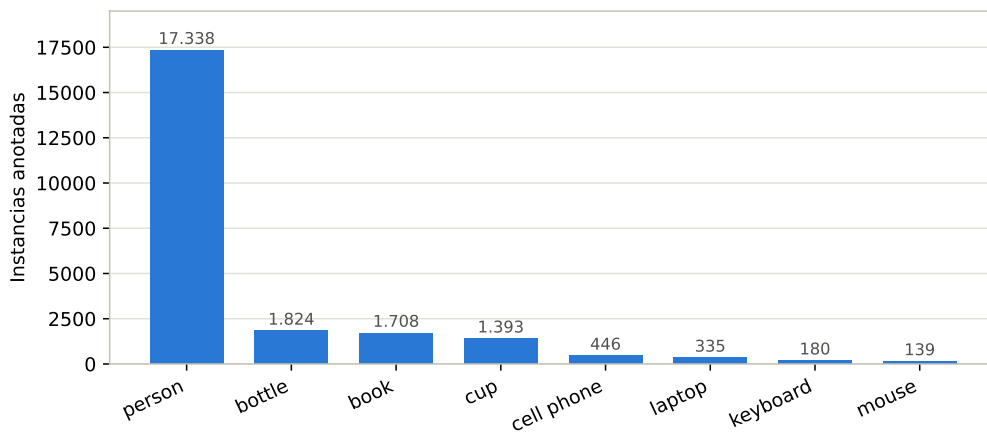


Figura 1: Distribución de instancias anotadas por clase en el subconjunto construido (5.000 imágenes de COCO 2017).

2.2. Particiones

El subconjunto se dividió en tres particiones disjuntas a nivel de imagen, con semilla fija: **entrenamiento** (60 %, 3.000 imágenes), destinada al ajuste de los pesos; **validación** (15 %, 750 imágenes), empleada para el monitoreo del entrenamiento y la selección del mejor punto de control; y **prueba** (25 %, 1.250 imágenes), reservada exclusivamente para la evaluación final, de modo que las métricas reportadas sobre ella no intervienen en ninguna decisión de entrenamiento. Se verificó que la distribución de clases se mantuviera aproximadamente proporcional entre particiones.

2.3. Conversión al formato YOLO de segmentación

El entrenador de Ultralytics requiere las anotaciones en formato YOLO de segmentación: un archivo de texto por imagen en el que cada línea describe una instancia como un índice de clase seguido de los vértices de su polígono, normalizados al rango $[0, 1]$. La conversión comprendió dos pasos:

1. **De máscaras a polígonos.** Las instancias de COCO cargadas por FiftyOne se representan como máscaras binarias; se convirtieron a polígonos con una tolerancia de simplificación de 2 píxeles, que reduce el número de vértices sin pérdida perceptible de fidelidad geométrica.
2. **Saneamiento de fragmentos degenerados.** Las máscaras multi-parte (objetos divididos por oclusión) y los objetos muy pequeños pueden producir, tras la simplificación, polígonos de menos de tres vértices, que el cargador de Ultralytics interpreta como cajas y rechaza. Estas líneas —aproximadamente el 3 % del total, fragmentos sin área significativa— se eliminaron de los archivos exportados; la componente principal de cada objeto conserva su polígono.

Como control de calidad se reconstruyeron las anotaciones a partir de los archivos exportados y se superpusieron sobre las imágenes correspondientes (Figura 2), validando la cadena completa de conversión: formato, normalización de coordenadas y asignación de índices de clase.



Figura 2: Verificación de la conversión: anotaciones reconstruidas desde los archivos YOLO exportados, superpuestas sobre tres imágenes de entrenamiento. Los polígonos coinciden con los contornos de los objetos y las etiquetas de clase son correctas.

3. Metodología

3.1. Arquitectura: YOLOv8n-seg

Se adoptó **YOLOv8-seg** [2], un modelo de segmentación de instancias de una sola etapa (*one-stage*) y libre de anclas (*anchor-free*), heredero de la familia YOLO [4]. Su arquitectura se compone de:

- un **backbone** convolucional (CSPDarknet con bloques C2f) que extrae mapas de características a múltiples escalas;
- un **neck** de tipo PAN-FPN que fusiona información semántica de alto nivel con detalle espacial de bajo nivel;
- una **cabeza desacoplada** que predice, para cada objeto, la caja delimitadora, la clase y un vector de coeficientes de máscara, junto con una rama que genera *prototipos* de máscara a resolución de imagen (esquema derivado de YOLACT [3]). La máscara final de cada instancia se obtiene como combinación lineal de los prototipos, lo que permite segmentar con costo computacional casi constante por imagen.

Se eligió la variante **nano** (3,26 millones de parámetros, 11,4 GFLOPs), la de menor capacidad de la familia, porque el requisito de tiempo real debe satisfacerse sobre la CPU de una computadora personal: es la variante con mejor relación desempeño–latencia para ese escenario.

Estrategia de transferencia. Se partió de los pesos preentrenados en COCO completo y se ajustaron *todos* los parámetros sobre el subconjunto de 8 clases (ajuste fino completo, sin congelamiento de capas). Dado que el dominio de origen y el de destino coinciden —las ocho clases pertenecen a COCO—, el preentrenamiento aporta representaciones ya adecuadas y el ajuste se concentra en especializar el modelo en el vocabulario reducido, lo que permite converger en pocas épocas con un conjunto de datos moderado.

3.2. Preprocesamiento y aumento de datos

Preprocesamiento (entrenamiento e inferencia): toda imagen se redimensiona a 640×640 píxeles mediante *letterbox* (se preserva la relación de aspecto y se rellena con bordes neutros) y las intensidades se normalizan al rango $[0, 1]$.

Aumento de datos (solo entrenamiento): amplía artificialmente la variabilidad del conjunto de entrenamiento y actúa como regularizador. Se empleó el esquema estándar de Ultralytics, que transforma de manera consistente las imágenes *y* sus polígonos de segmentación. La Tabla 1 resume las transformaciones y su justificación; la Figura 3 ilustra un lote de entrenamiento real con *mosaic*. Se omitieron el volteo vertical, las rotaciones y *mixup*, que generarían poses inverosímiles para el dominio sin beneficio esperado.

Tabla 1: Esquema de aumento de datos empleado durante el entrenamiento.

Transformación	Valor	Justificación
<i>Mosaic</i>	1,0	Compone cuatro imágenes en una: expone al modelo a objetos en contextos y escalas atípicos. Se desactiva en las últimas 10 épocas (<code>close_mosaic=10</code>).
Volteo horizontal	$p = 0,5$	Los objetos del dominio no tienen quiralidad relevante; duplica la variabilidad de pose.
Traslación	$\pm 10\%$	Robustez ante encuadres imperfectos, frecuentes en una cámara web.
Escala	$\pm 50\%$	Robustez ante variaciones de distancia objeto-cámara, críticas para la demostración en vivo.
Perturbación HSV	$h=0,015; s=0,7; v=0,4$	Robustez ante cambios de iluminación y balance de color entre las escenas de COCO y la cámara del usuario.

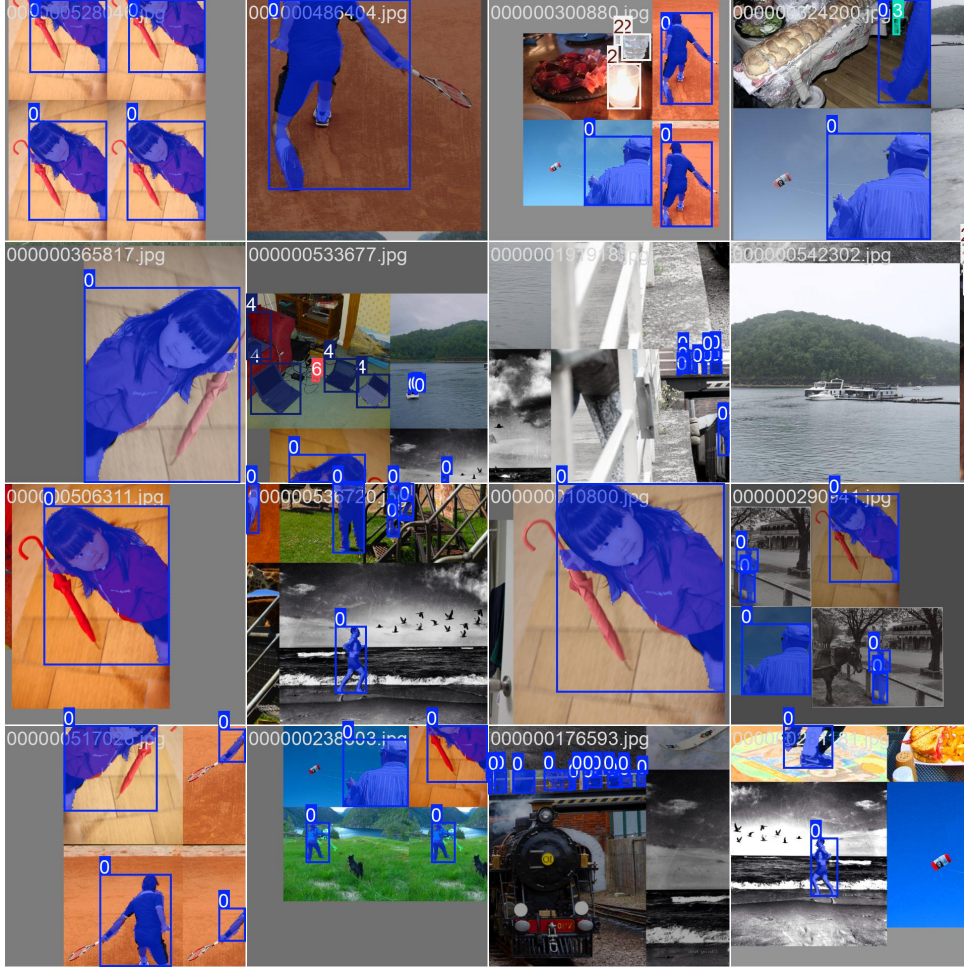


Figura 3: Lote de entrenamiento real con aumento *mosaic*: cada muestra compone cuatro imágenes con sus máscaras transformadas de manera consistente.

3.3. Configuración del entrenamiento

El entrenamiento se ejecutó en un servidor con GPU NVIDIA GeForce RTX 5090 (32 GB), con Ultralytics 8.4.89, PyTorch 2.12.1 (CUDA 13.0) y Python 3.12. La Tabla 2 resume la configuración. El tamaño de lote se fijó explícitamente en 32 y los procesos de precarga de datos en 4, valores elegidos para acotar el consumo de memoria del sistema en el servidor compartido. El optimizador AdamW [6] y su tasa de aprendizaje fueron seleccionados por la heurística automática de Ultralytics. El mejor punto de control se seleccionó por el mAP@50-95 de máscaras sobre la partición de validación, alcanzado en la última época (0,450).

Tabla 2: Configuración del entrenamiento.

Parámetro	Valor
Modelo base	yolov8n-seg.pt (preentrenado en COCO)
Épocas	60 (early stopping con paciencia 15; completó las 60)
Resolución de entrada	640 × 640 (letterbox)
Tamaño de lote	32
Optimizador	AdamW (selección automática), $lr_0 = 8,33 \times 10^{-4}$, momento 0,9
Decaimiento de pesos	5×10^{-4}
Precisión mixta (AMP)	Sí
Semilla	42 (modo determinista)
Duración total	86,5 minutos

4. Experimentos y resultados

4.1. Protocolo de evaluación

Las métricas siguen el protocolo estándar de COCO. El **IoU** (*Intersection over Union*) mide el solapamiento entre una predicción P y su anotación de referencia G :

$$IoU(P, G) = \frac{|P \cap G|}{|P \cup G|}, \quad (1)$$

y se aplica tanto a cajas delimitadoras como a máscaras. Una predicción es un verdadero positivo si su IoU con una anotación de la misma clase supera un umbral dado. La **AP** (*Average Precision*) de una clase es el área bajo su curva precisión–recall, y el **mAP** es su promedio sobre las clases. Se reportan las dos variantes estándar: **mAP@50** (umbral de IoU de 0,5, criterio laxo) y **mAP@50-95** (promedio sobre umbrales de 0,5 a 0,95 en pasos de 0,05, criterio estricto que premia la precisión de la localización), calculadas por separado para cajas (detección) y máscaras (segmentación). Toda la evaluación de esta sección se realiza sobre la partición de **prueba** (1.250 imágenes, 7.117 instancias).

4.2. Curvas de aprendizaje

La Figura 4 muestra la evolución del entrenamiento. Las pérdidas de caja y de segmentación decrecen de forma sostenida tanto en entrenamiento como en validación, sin divergencia entre ambas —no se observa sobreajuste. Que la pérdida de validación se ubique por debajo de la de entrenamiento es esperable: la pérdida de entrenamiento se calcula sobre lotes fuertemente aumentados (*mosaic*), más difíciles que las imágenes naturales de validación. El mAP de validación crece de forma monótona (mAP@50 de cajas: de 0,27 en la primera época a 0,70 en la última) y presenta el repunte característico en las últimas 10 épocas, cuando *mosaic* se desactiva y el modelo termina de ajustarse sobre imágenes de estadística natural (mAP@50-95 de máscaras: de 0,422 en la época 50 a 0,450 en la 60).

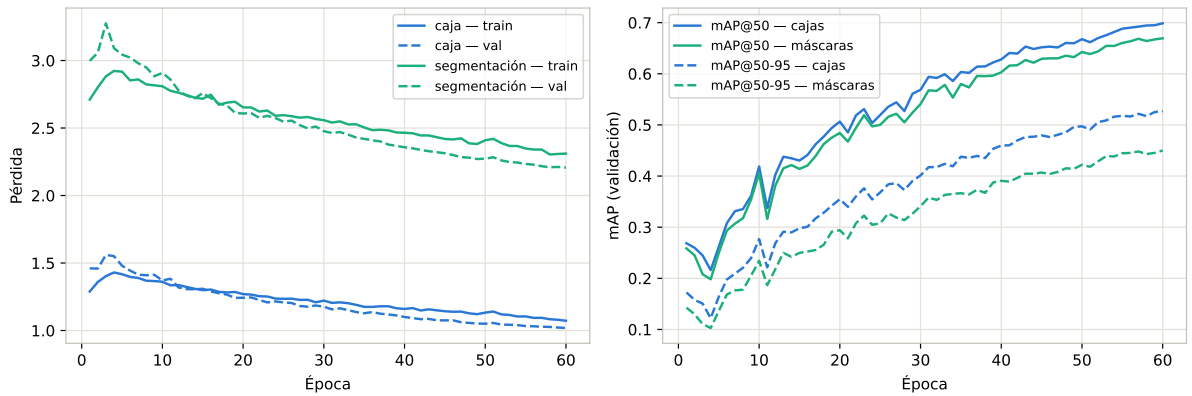


Figura 4: Evolución del entrenamiento. Izquierda: pérdidas de caja y de segmentación en entrenamiento (línea continua) y validación (línea discontinua). Derecha: mAP@50 y mAP@50-95 sobre la partición de validación, para cajas y máscaras.

4.3. Métricas sobre la partición de prueba

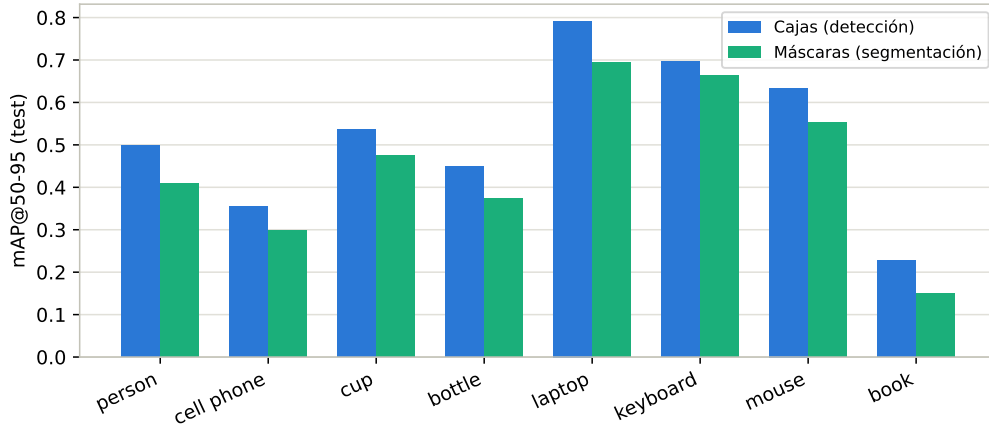
La Tabla 3 resume las métricas globales del modelo ajustado sobre la partición de prueba y la Tabla 4 las desglosa por clase; la Figura 5 las presenta gráficamente.

Tabla 3: Métricas globales sobre la partición de prueba (1.250 imágenes, 7.117 instancias).

Métrica	Cajas (detección)	Máscaras (segmentación)
Precisión media	0,767	0,752
Recall medio	0,626	0,609
mAP@50	0,705	0,675
mAP@50-95	0,524	0,452

Tabla 4: Métricas por clase sobre la partición de prueba.

Clase	Instancias	Cajas		Máscaras	
		mAP@50	mAP@50-95	mAP@50	mAP@50-95
person	5.448	0,711	0,499	0,686	0,408
cell phone	103	0,518	0,354	0,477	0,299
cup	415	0,747	0,536	0,733	0,476
bottle	527	0,657	0,449	0,627	0,374
laptop	94	0,927	0,792	0,905	0,694
keyboard	40	0,882	0,697	0,882	0,663
mouse	35	0,808	0,634	0,783	0,553
book	455	0,393	0,228	0,306	0,149
Todas	7.117	0,705	0,524	0,675	0,452

**Figura 5:** mAP@50-95 por clase sobre la partición de prueba, para cajas y máscaras.

Del desglose surgen tres observaciones:

- **Mejores clases: laptop, keyboard y mouse** (mAP@50-95 de cajas de 0,792, 0,697 y 0,634). Son objetos rígidos, de contorno regular y que suelen aparecer a escala considerable y poco ocluidos, condiciones favorables tanto para la localización como para la segmentación.
- **Peor clase: book** (0,228 en cajas, 0,149 en máscaras). En COCO los libros aparecen mayoritariamente apilados o en estanterías, en instancias pequeñas, parcialmente visibles y con límites ambiguos entre ejemplares contiguos; la propia anotación de referencia es inconsistente en esos casos. Es una dificultad conocida de esta categoría.
- **Las máscaras puntúan sistemáticamente por debajo de las cajas** (0,452 frente a 0,524 global). El criterio de IoU a nivel de píxel es más estricto que el de solapamiento de

rectángulos, en particular para contornos complejos como los de **person**, donde la diferencia es máxima (0,408 frente a 0,499).

4.4. Matriz de confusión y curvas precisión–recall

La matriz de confusión normalizada (Figura 6) muestra que el modelo prácticamente **no confunde clases entre sí**: la mayor confusión inter-clase es de apenas 0,02. El modo de error dominante es la **omisión** (objetos reales clasificados como fondo), que afecta sobre todo a **book** (64 % de las instancias omitidas) y **cell phone** (53 %), y en mucha menor medida a **keyboard** (10 %) y **laptop** (12 %). Este patrón es coherente con el análisis por clase: el vocabulario reducido hace que el problema de clasificación sea sencillo, y el desafío restante es la detección de instancias pequeñas u ocluidas. Las curvas precisión–recall (Figura 7) muestran el mismo ordenamiento de clases en ambas tareas.

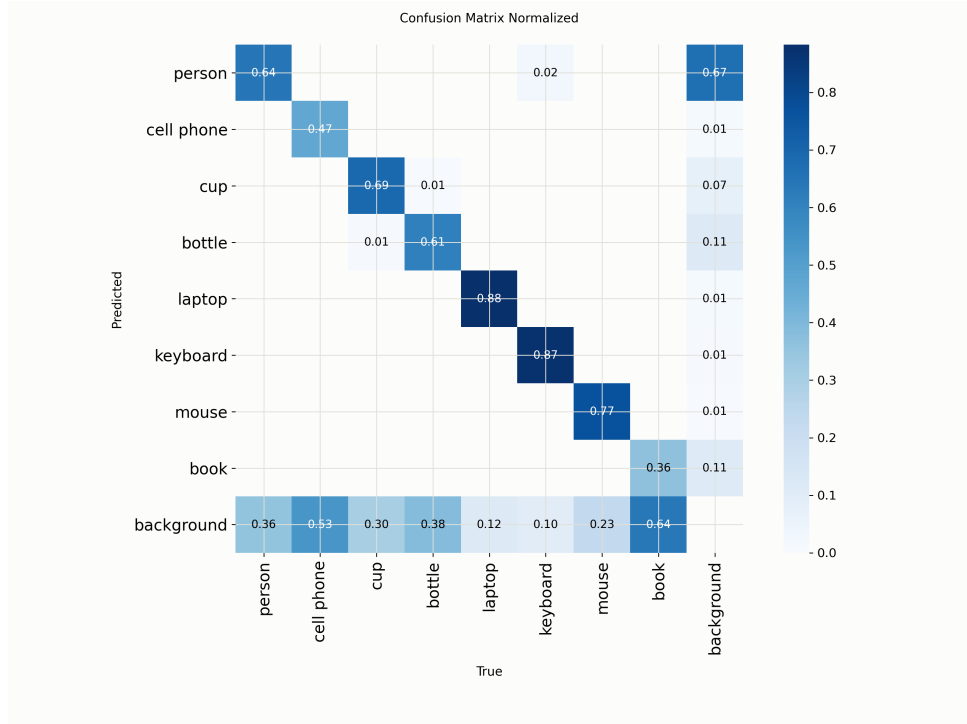
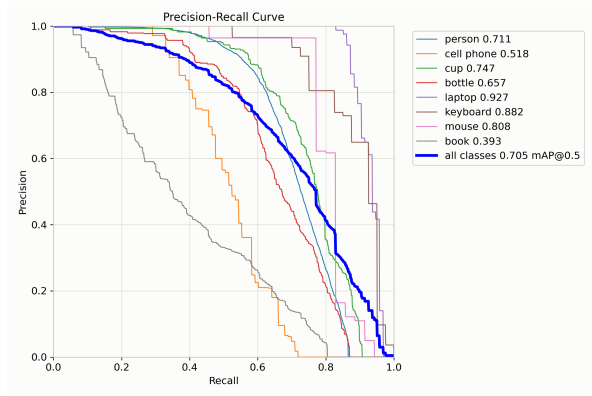
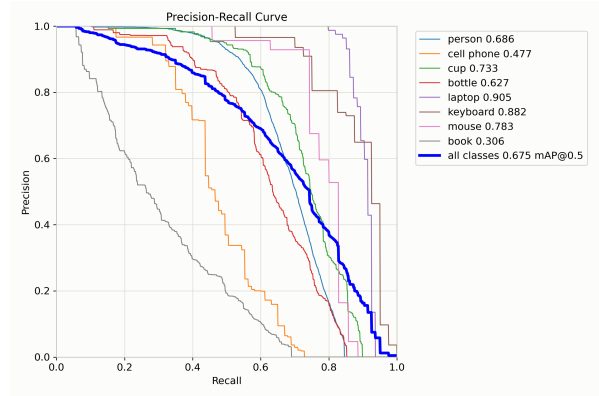


Figura 6: Matriz de confusión normalizada por columnas sobre la partición de prueba (columnas: clase real; filas: clase predicha). La diagonal concentra las predicciones correctas; la última fila corresponde a objetos reales no detectados (omisiones).



(a) Cajas (detección).



(b) Máscaras (segmentación).

Figura 7: Curvas precisión–recall por clase sobre la partición de prueba, con umbral de IoU de 0,5. El área bajo cada curva es la AP@50 de la clase.

4.5. Comparación con el modelo de referencia

Para cuantificar el aporte del ajuste fino se evaluó el modelo **preentrenado original** (yolov8n-seg.pt, 80 clases) sobre la *misma* partición de prueba. Para ello, la partición se exportó adicionalmente con el vocabulario completo de 80 clases de COCO, de modo que los índices de clase de las etiquetas coincidieran con los del modelo, y de los resultados se extrajeron las AP de las ocho clases de interés. La Tabla 5 y la Figura 8 presentan la comparación sobre la métrica principal (mAP@50-95 de máscaras).

Tabla 5: Comparación del mAP@50-95 de máscaras por clase, sobre la partición de prueba, entre el modelo preentrenado de referencia y el modelo ajustado.

Clase	Baseline preentrenado	Ajustado (este trabajo)	Mejora
person	0,324	0,408	+0,084
cell phone	0,192	0,299	+0,107
cup	0,334	0,476	+0,142
bottle	0,269	0,374	+0,105
laptop	0,579	0,694	+0,115
keyboard	0,431	0,663	+0,232
mouse	0,553	0,553	0,000
book	0,058	0,149	+0,091
Promedio	0,342	0,452	+0,110

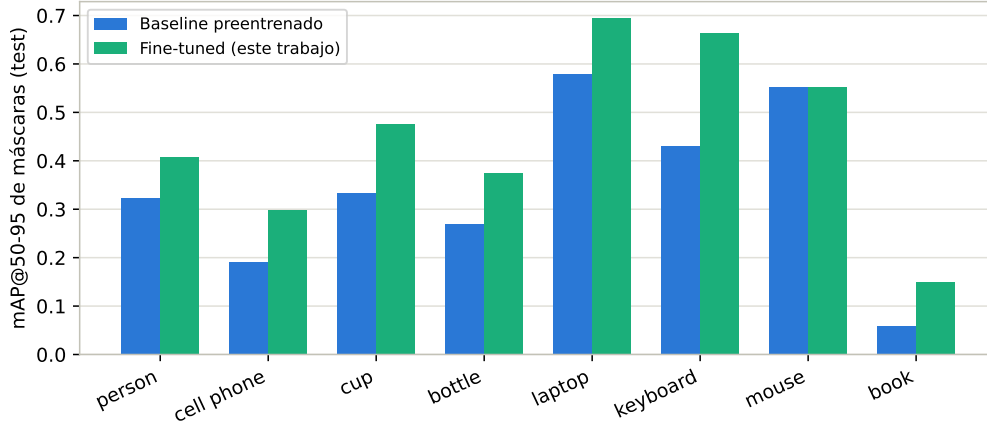


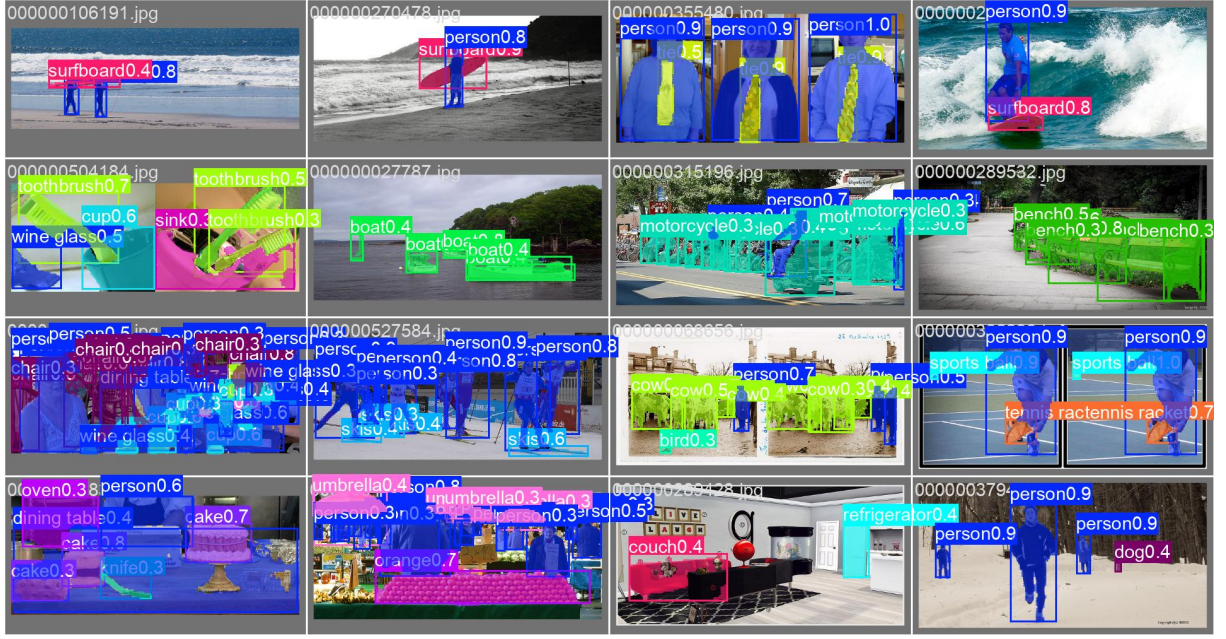
Figura 8: Aporte del ajuste fino por clase: mAP@50-95 de máscaras sobre la partición de prueba.

El ajuste fino mejora la métrica global de máscaras de 0,342 a 0,452 (+32 % relativo) y la de cajas de 0,403 a 0,524 (+30 % relativo). La mejora es generalizada y máxima en **keyboard** (+0,232); la única clase sin mejora es **mouse**, cuyo desempeño de partida ya era alto y que cuenta con muy pocos ejemplos de entrenamiento para especializar.

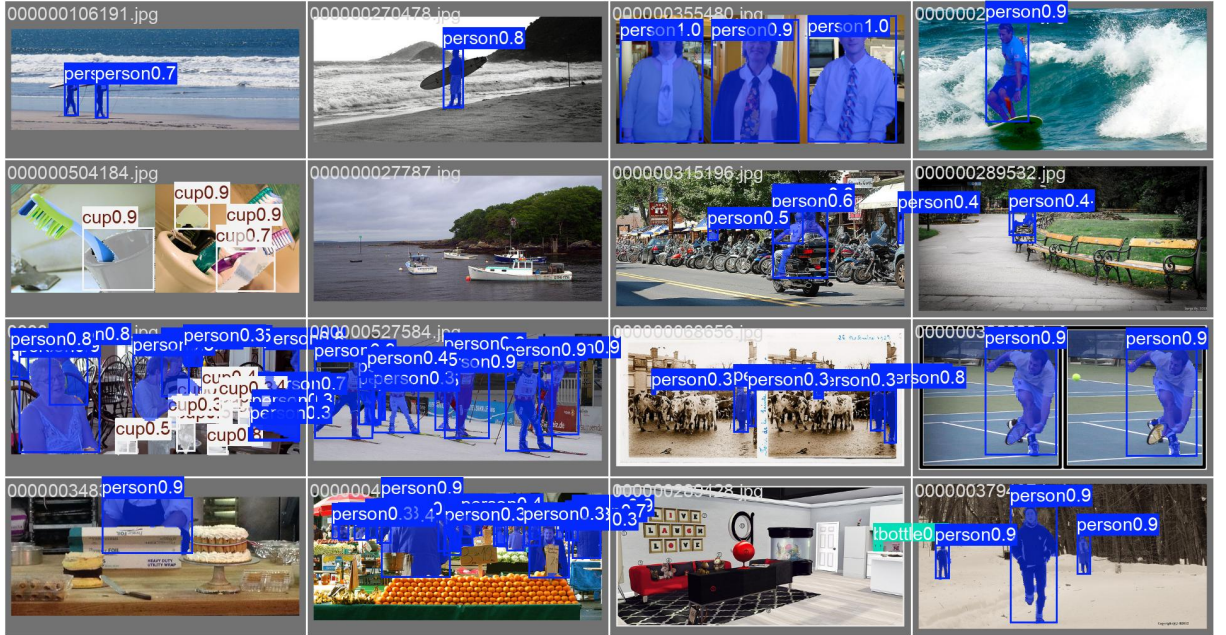
Consideración metodológica. La comparación es conservadora respecto del baseline: el modelo preentrenado resuelve un problema más difícil (80 clases posibles frente a 8), por lo que parte de la mejora observada se explica por la reducción del vocabulario además de por el ajuste de los pesos. La comparación cuantifica, en definitiva, el beneficio conjunto de especializar el modelo en el dominio de aplicación.

4.6. Análisis cualitativo

La Figura 9 contrasta las predicciones de ambos modelos sobre el mismo lote de 16 imágenes de la partición de prueba. El efecto de la especialización es visible: el baseline dispersa sus predicciones entre categorías fuera del alcance del sistema (**surfboard**, **tie**, **boat**, **bench**, **cow**), mientras que el modelo ajustado concentra sus detecciones en las ocho clases de interés con confianzas más altas y máscaras ajustadas al contorno. Se aprecian también los modos de error residuales identificados en la matriz de confusión: personas pequeñas en escenas concurridas y libros en estanterías que no son detectados.



(a) Modelo preentrenado de referencia (80 clases).



(b) Modelo ajustado (8 clases, este trabajo).

Figura 9: Predicciones de ambos modelos sobre el mismo lote de imágenes de la partición de prueba (umbral de confianza de la rutina de validación).

5. Sistema de inferencia en tiempo real

5.1. Implementación

El sistema de inferencia (`src/demo_webcam.py`) captura video de una cámara web —o de un archivo— mediante OpenCV, ejecuta el modelo sobre cada cuadro y superpone en pantalla las cajas, las máscaras de instancia, las etiquetas de clase con su confianza y la tasa de cuadros por segundo (FPS), esta última suavizada con una media móvil exponencial para una lectura estable. Los parámetros operativos (fuente de video, umbral de confianza, resolución de inferencia) son configurables por línea de comandos.

Dado que la computadora de demostración no dispone de GPU con CUDA, se evaluó además la **optimización del modelo con OpenVINO** [7]: los pesos entrenados se exportaron al formato de representación intermedia de Intel (`src/export_openvino.py`), especializado para inferencia sobre CPU y gráficos integrados Intel.

5.2. Desempeño medido

La Tabla 6 resume las mediciones de latencia en las tres configuraciones evaluadas. En el servidor, la rutina de validación reporta 5,1 ms por imagen (~ 198 FPS teóricos), holgadamente en tiempo real. En la computadora personal (CPU Intel Core Ultra 5 226V), el modelo original en formato PyTorch opera a 17 FPS, y la variante OpenVINO lo duplica hasta 35 FPS, superando el umbral convencional de tiempo real (24–30 FPS) sobre hardware de consumo sin aceleración dedicada.

Tabla 6: Velocidad de inferencia medida (resolución de entrada de 640 píxeles).

Plataforma	Formato del modelo	Latencia	FPS
Servidor, GPU RTX 5090	PyTorch (.pt)	5,1 ms/imagen	~ 198
PC personal, CPU Core Ultra 5	PyTorch (.pt)	~ 59 ms/cuadro	17
PC personal, CPU Core Ultra 5	OpenVINO	~ 29 ms/cuadro	35

La Figura 10 muestra el sistema en operación sobre una escena real de escritorio. En ambas capturas el modelo detecta y segmenta correctamente los cinco objetos presentes (**person**, **book**, **mouse**, **bottle**), con máscaras que siguen el contorno de los objetos —incluida la silueta de la persona y el libro abierto sostenido en la mano— bajo la iluminación no controlada de la escena. El indicador de FPS en pantalla evidencia la ganancia de la optimización: 17,2 FPS con el modelo PyTorch frente a 35,6 FPS con OpenVINO, sobre la misma escena y el mismo hardware.

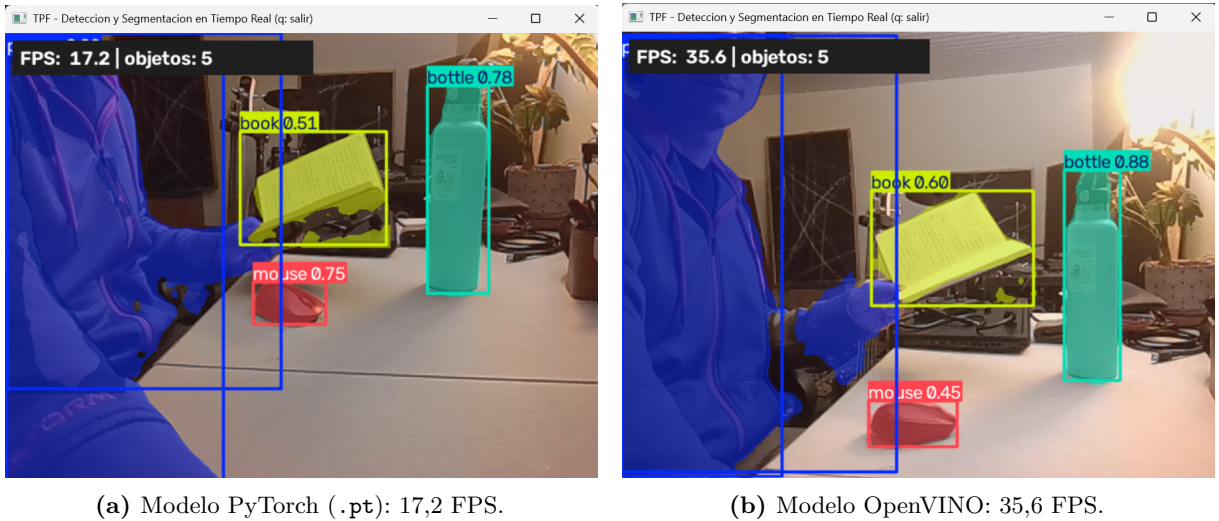


Figura 10: Demostración del sistema en tiempo real sobre cámara web (CPU Intel Core Ultra 5 226V, sin GPU dedicada). Misma escena, cinco objetos detectados y segmentados en ambos casos.

6. Conclusiones

Se desarrolló un sistema completo de visión por computadora que cubre el ciclo íntegro de un proyecto de aprendizaje profundo aplicado: construcción de un conjunto de datos supervisado (5.000 imágenes y 23.363 instancias de COCO 2017, con conversión y saneamiento de anotaciones de segmentación), ajuste fino de un modelo de segmentación de instancias YOLOv8n-seg,

evaluación rigurosa con las métricas estándar del área e implementación de la inferencia en tiempo real sobre hardware de consumo. Del trabajo se extraen las siguientes conclusiones:

1. **La transferencia de aprendizaje es efectiva y eficiente para especializar un detector-segmentador en un vocabulario reducido.** Con 3.000 imágenes de entrenamiento y 60 épocas (86,5 minutos de GPU), el ajuste fino elevó el mAP@50-95 promedio sobre las ocho clases de 0,403 a 0,524 en detección y de 0,342 a 0,452 en segmentación (+30 % y +32 % relativos frente al modelo preentrenado evaluado sobre las mismas imágenes), con mejoras en siete de las ocho clases.
2. **El desempeño por clase está gobernado por las características visuales y la calidad de anotación de cada categoría.** Los objetos rígidos y poco ocluidos (`laptop`, `keyboard`, `mouse`) superan 0,63 de mAP@50-95 de cajas, mientras que `book` —instancias pequeñas, apiladas y de anotación ambigua en COCO— queda en 0,228. La matriz de confusión muestra que el error dominante no es la confusión entre clases (máximo 0,02) sino la omisión de instancias pequeñas u ocluidas, lo que señala que el margen de mejora está en el recall y no en la discriminación.
3. **La segmentación de instancias en tiempo real es viable sobre una CPU de consumo.** La variante nano de YOLOv8-seg, exportada a OpenVINO, alcanzó 35 FPS sostenidos en una computadora personal sin GPU dedicada —el doble que el formato PyTorch original—, validando la combinación de una arquitectura compacta con optimización específica del hardware de despliegue como estrategia para llevar modelos de segmentación a aplicaciones interactivas.
4. **El pipeline resultó reproducible de extremo a extremo:** la selección de datos, las particiones y el entrenamiento están gobernados por semillas fijas, y la demostración en vivo confirmó que el desempeño cuantitativo medido sobre COCO se traslada a escenas reales no controladas, con máscaras que siguen fielmente el contorno de los objetos.

Anexos. Acompañan a este informe los notebooks ejecutados del pipeline (`01_dataset`, `02_entrenamiento`, `03_evaluacion`), el código fuente del sistema de inferencia (`src/`) y los pesos del modelo entrenado (`models/yolov8n_seg_best.pt`).

Referencias

- [1] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár y C. L. Zitnick, “Microsoft COCO: Common Objects in Context”, en *Proc. European Conference on Computer Vision (ECCV)*, 2014.
- [2] G. Jocher, A. Chaurasia y J. Qiu, “Ultralytics YOLOv8”, 2023. Disponible en: <https://github.com/ultralytics/ultralytics>
- [3] D. Bolya, C. Zhou, F. Xiao y Y. J. Lee, “YOLACT: Real-time Instance Segmentation”, en *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [4] J. Redmon, S. Divvala, R. Girshick y A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection”, en *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [5] B. E. Moore y J. J. Corso, “FiftyOne”, GitHub, 2020. Disponible en: <https://github.com/voxel51/fiftyone>

- [6] I. Loshchilov y F. Hutter, “Decoupled Weight Decay Regularization”, en *Proc. International Conference on Learning Representations (ICLR)*, 2019.
- [7] Intel Corporation, “OpenVINO Toolkit: Open-source toolkit for optimizing and deploying AI inference”. Disponible en: <https://docs.openvino.ai>